

KiberEduAz

Consolidated Security Audit

Findings for Engineering — Merged from three independent review passes

REPOSITORY	1brah1m0f/KiberEduAz
STACK	NestJS API · Next.js (App Router) frontend · Prisma/Supabase Postgres · Supabase Auth · Render + Vercel hosting
AUDIT DATE	2026-09-11
METHOD	Full static review (source, git history, Prisma schema/migrations, deployment config, npm audit against both lockfiles). Read-only commands only — no files modified, no packages installed, no live systems probed.
SOURCES MERGED	Production-Readiness Security Audit (49-phase) · Toplu Güvenlik Denetimi (23-item) · Security Review (static assessment) — reconciled into one authoritative finding set below.

3	7	5	3
CRITICAL	MEDIUM	LOW	NEEDS VERIFICATION

VERDICT

NOT READY FOR PRODUCTION

Two confirmed critical authorization flaws exploitable by any approved teacher account, one critical dependency advisory, one potentially-critical unverified database exposure, and no account-deletion path on a platform aimed at minors. All P0 fixes are small — the project can realistically reach **Ready with Conditions** after P0 + P1 are closed.

1 · Executive Summary

Overall posture: the engineering baseline is above average — deny-by-default guards, server-side answer grading, strict DTO validation, parameterized queries everywhere, provider-managed auth, and zero committed secrets. That baseline is undermined by **one systematic architectural gap — role checks without ownership checks** — on every content-authoring endpoint, plus one critical dependency advisory and several provider-side controls that cannot be confirmed from source alone.

Top risks, in order

- Any approved teacher can read and rewrite any other teacher's content.** This includes dumping every answer key on the platform and silently altering live exam questions. Reachable via a single admin approval of a self-requested teacher account — see `F-01` / `F-02`.
- The deployed Next.js version carries a critical public advisory** (`F-03`), on the internet-facing frontend.
- The Supabase database's exposure surface is unverified.** No RLS policies exist anywhere in the repo, the backend connects with the RLS-bypassing `postgres` role, legacy tables from a prior project are still present in the instance, and the public `publishable` key is committed by design. If the Supabase Data API (PostgREST) is enabled and any public table lacks RLS, that table is anonymously readable today — see `F-10`.

Why this matters together, not just separately: teacher status is *self-requested* (`POST /profiles/me/request-teacher`) and gated by a single admin click. That one approval is the only thing standing between a student account and full read/write access to the entire content corpus. Fixing F-01/F-02 removes that blast radius; leaving the self-request flow as-is without the ownership fix means one social-engineered approval equals total content compromise.

Severity breakdown

SEVERITY	COUNT	IDS
CRITICAL	3	F-01 (BOLA / answer-key leak), F-02 (BOLA / paths & modules), F-03 (critical <code>next</code> dependency advisory)
MEDIUM	7	F-04 (points race), F-05 (escalation blast radius), F-06 (open redirect), F-07 (forwarded-host trust), F-08 (missing frontend headers), F-09 (no account deletion), F-10 (DB/RLS exposure — unverified, potentially critical)
LOW	5	F-11 (CORS error handling), F-12 (constraint-name leak), F-13 (rate-limit granularity), F-14 (draft visibility), F-15 (email local-part disclosure)
NEEDS VERIFICATION	3	Backups/PITR, Supabase Auth limits & token lifetimes, live HTTPS/security-header behavior

Note on reconciliation: two earlier passes (the 23-item and the narrative review) flagged a missing `@Min(0)` validator on `orderIndex`. The most recent pass confirms this validator is now present (`content.dto.ts:58,71,101,110`) — it has been removed from the open findings below and is listed under "Verified Secure." This is the only material delta between passes; every other finding is corroborated across all three reviews.

2 · Architecture & Attack Surface

Frontend

- Next.js 16 (App Router), React 19, TypeScript, Tailwind 4, react-markdown 10
- Deployed on Vercel (inferred from CORS config; hosting config itself not in repo)
- Session via `@supabase/ssr` cookies; `proxy.ts` middleware revalidates with `getUser()`

Backend

- NestJS 11 REST API under `/api/v1`
- Global `Helmet`, `ThrottlerGuard` (120 req/min), `JwtAuthGuard` + `RolesGuard` (deny-by-default, only `/health` public)
- Global `ValidationPipe` (whitelist + forbidNonWhitelisted + transform)

Internet-facing surface

- Next.js pages behind middleware, plus two pre-auth route handlers: `GET /auth/callback` and `POST /auth/signout`
- NestJS API: `health` (public), `rooms`, `paths`, `modules`, `progress`, `profiles`, `classes`, `leaderboard`, `notifications`, `admin` — all require a verified JWT except `/health`
- Supabase Auth endpoints (provider-managed, called directly from the browser)
- Supabase Data API (PostgREST) — enabled/disabled state is **unknown from source** and reachable with the public publishable key if enabled

No SSRF surface exists (no server-side fetch of user-supplied URLs). Untrusted input crosses into trusted logic at: API request bodies/params (globally validated), the auth-callback query string (partially validated — see F-06), and forwarded headers trusted for redirect construction (see F-07).

Auth & Database

- Supabase Auth (hosted); backend verifies JWTs via `jose` against JWKS, pinning `issuer` and `aud=authenticated`
- Supabase Postgres 17 via Prisma 6, accessed through the pooler as the `postgres` role (bypasses RLS)
- **No RLS policies exist anywhere in the repository**

Deployment & scope

- Render (free plan) auto-deploy on commit; no CI/CD workflows in repo
- No object storage, no payments, no SMS, no AI/LLM, no webhooks, no mobile app
- No backend test framework; minimal frontend test script

3 · Critical Findings

F-01 · CONFIRMED · Broken Object-Level Authorization (BOLA)

Any approved teacher can read and modify any room in the catalog — including answer keys

WHERE

backend/src/catalog/rooms.controller.ts:36-39,80,90,96-97 · rooms.service.ts:147,180,214,263

WHAT'S WRONG

- findByIdForAuthor(id) serves the full answer key (isCorrect , explanation , acceptedAnswers) for **any** room id, including other teachers' drafts and published rooms — reached via GET /rooms/:id/edit , which checks role only, never authorship.
- update() rewrites any room regardless of author; a teacher can inject content into a **published** room owned by someone else. Only the status field is protected from teachers.
- upsertTask / removeTask can add, wholesale-overwrite, or delete tasks belonging to any room; the nested taskId is also never checked against its parent :id (see F-14-adjacent bug).
- Room listings for teachers return every draft in the system, not just their own.

IMPACT

A single approved teacher account can dump every answer key on the platform, silently alter exam/content questions (including which option is correct), or destroy the catalog. Because teacher status is self-requested and gated by one admin click, social-engineering a single approval is sufficient for full content compromise.

FIX

Thread the requesting user into every service call and enforce room.createdById === user.id || role === ADMIN — mirror the pattern already implemented correctly in ClassesService.assertCanManage . Validate nested taskId → roomId ownership in the same pass. One shared guard/helper covers all ~7 affected endpoints. Complexity: Small. Breaking risk: Low (teachers only lose cross-editing access they should never have had).

F-02 · CONFIRMED · Broken Object-Level Authorization (BOLA)

Same missing ownership model on learning paths and modules

WHERE

backend/src/catalog/paths.controller.ts:36,60

WHAT'S WRONG

PATCH /paths/:id and PATCH /modules/:id don't even receive the caller as a parameter (updatePath(id, dto)) — there is nothing to check. Any approved teacher can rename, restructure, or delete any path or module, and create modules under any path.

FIX

Same shared ownership guard as F-01, applied at the curriculum-structure level.

F-03 · CONFIRMED · Dependency Security

Critical public advisory on the deployed Next.js version

WHERE

frontend/package.json (next ^16) and its lockfile

WHAT'S WRONG

- Frontend (critical):** next 16.0.0–16.3.2 carries a critical advisory; the fix ships in 16.3.4, outside the currently declared range.
- Frontend (high):** sharp < 0.35.4 , js-yaml , nanoid < 3.3.18 .
- Backend (high/moderate):** 9 high-severity advisories (including deepmerge-ts , qs) and 1 moderate. Lockfiles are committed for both packages, so the audit is reproducible.

FIX

Upgrade next@16.3.4 and sharp first; run the full build and a smoke test (the patch is outside the currently declared semver range). Then run npm audit fix for non-breaking backend advisories, and schedule any major-version bumps separately. Complexity: Small–Medium. Breaking risk: Medium.

4 · Medium Findings

F-04 · CONFIRMED · Race Condition / Business-Logic Integrity Double points payout on concurrent correct submissions WHERE backend/src/progress/progress.service.ts:40-46, 277, 286 WHAT'S WRONG The <code>alreadySolved</code> check runs outside the write transaction (TOCTOU). Two concurrent correct submissions can both observe "not yet solved" and both create an <code>answerAttempt</code> + <code>pointsLedger</code> entry, double-paying <code>question.points</code>. The same pattern affects the first-completion bonus. FIX Add a partial unique index on <code>(profileId, questionId) WHERE is_correct</code> in Prisma, and derive payout idempotently from the ledger inside the transaction (upsert-with-conflict) rather than from a pre-check. <i>Complexity: Small. One migration.</i>
F-05 · CONFIRMED · Privilege-Escalation Blast Radius Self-requested teacher role is the only gate in front of F-01/F-02 WHERE backend/src/profiles/profiles.controller.ts:28-30, profiles.service.ts (requestTeacher) WHAT'S WRONG Any student can self-nominate as a teacher; only a single admin approval separates that request from the powers described in F-01/F-02. This amplifies those findings from "abuse by a trusted peer" to "abuse by anyone willing to ask." FIX Keep the approval gate, but make it matter: once F-01/F-02 are fixed, approved teachers only ever touch their own content. Consider an explicit probation scope for newly approved teachers as defense-in-depth.
F-06 · CONFIRMED · Open Redirect Backslash bypass in the post-auth redirect allowlist WHERE frontend/src/app/auth/callback/route.ts:22-23,49 WHAT'S WRONG <code>safePath</code> rejects values starting with <code>//</code> but not <code>/\evil.com</code>. Browsers normalize backslashes to forward slashes in URLs, turning this into a protocol-relative redirect to an attacker-controlled origin — reachable via a crafted email link. FIX Reject any <code>next</code> value whose first segment contains a backslash, or allowlist with a pattern that also excludes <code>\</code>. Add a regression test for backslash-variant payloads.
F-07 · LIKELY — requires runtime verification · Forwarded-Header Trust Redirect origin trusts <code>x-forwarded-host</code> WHERE frontend/src/app/auth/callback/route.ts:10-15 WHAT'S WRONG The redirect origin is built from <code>x-forwarded-host</code> when present. This is safe on Vercel today (the platform controls that header), but becomes exploitable the moment the app sits behind any proxy that forwards client-supplied headers unfiltered. FIX Pin the redirect origin to an explicit <code>NEXT_PUBLIC_SITE_URL</code> environment variable in production instead of trusting forwarded headers.
F-08 · CONFIRMED · Missing Security Headers Frontend ships with zero security headers WHERE frontend/next.config.ts:1-6 (empty) WHAT'S WRONG No CSP, <code>X-Frame-Options</code> / <code>frame-ancestors</code>, <code>Referrer-Policy</code>, or <code>Permissions-Policy</code> on the user-facing app. The backend has Helmet, but that doesn't cover page responses. Today's markdown rendering is safe, but this leaves no defense-in-depth for future content-editing surfaces. FIX Add a <code>headers()</code> block in <code>next.config.ts</code>. Roll out CSP as <code>Content-Security-Policy-Report-Only</code> first, review violation reports, then enforce. Confirm HSTS at the Vercel platform layer.

No way for a user — or an admin — to delete an account

WHERE

No delete endpoint exists in `backend/src`; only `frontend/src/app/auth/signout/route.ts` (sign-out, not deletion)

WHAT'S WRONG

User data spans `Profile`, `AnswerAttempt`, `PointsLedger`, `ClassMembership`, and `UserStats`, plus the Supabase auth user itself — none of it can currently be purged. This blocks any data-subject deletion request, which is especially sensitive for a platform whose target audience is minors.

FIX

Implement soft-delete with a 30-day restore window, followed by cascading purge across all tables and a Supabase Admin API `deleteUser` call. Document any retention exceptions explicitly. *Complexity: Medium/Large. Needs a design/retention sign-off.*

Supabase RLS / Data API posture is unknown and cannot be proven from source

WHERE

No RLS policy definitions anywhere in the repo; `backend/.env.example` confirms the `postgres` role (bypasses RLS); `backend/prisma/manual/drop_legacy_tables.sql` proves legacy tables from a prior (tourism) project still exist in the same Supabase instance.

WHY IT MATTERS

The frontend never talks to Postgres directly — all traffic goes through the NestJS API, which is correctly authorized once F-01/F-02 are fixed. But the app's *publishable* Supabase key is public by design (committed in `frontend/.env.example:2-3`). If Supabase's Data API (PostgREST) is enabled for this project and any public-schema table — including the legacy ones — lacks a default-deny RLS policy, that table is readable (possibly writable) anonymously, by anyone with the published key, completely bypassing the API layer.

VERIFY

Supabase Dashboard → Database → Policies, for every public-schema table. Or directly:

```
curl 'https://okyhjpywngmportlzmxo.supabase.co/rest/v1/<table>?select=*' -H 'apikey: <publishable_key>'
```

— data returned means it's exposed.

FIX

Enable RLS with a default-deny policy on every public-schema table, or disable the Data API entirely if the app never needs it (it doesn't, today). Drop or lock the legacy tables regardless. *Complexity: Small (dashboard-only).*

5 · Low-Severity & Informational Findings

F-11 · CONFIRMED · CORS Rejected origins throw instead of returning a clean CORS denial backend/src/main.ts:29 — callback(new Error(...)) aborts the response at the network layer rather than returning a proper CORS-denied response. No security impact (the deny still holds), but it breaks clean error observability. Fix: use callback(null, false) and log server-side.	
F-12 · CONFIRMED · Information Disclosure Database constraint name leaks in conflict errors backend/src/common/prisma-exception.filter.ts:29-33 — a Prisma P2002 conflict includes the constraint target (column names) in the response body. Minor internal-schema disclosure. Fix: return a generic conflict message; log the target server-side only.	
F-13 · CONFIRMED · Rate-Limit Granularity One global rate limit for every route backend/src/app.module.ts:30 — a single global 120 req/min applies everywhere; there's no tighter limit on answer submission, leaderboard reads, or profile updates, which makes slow brute-forcing of short-answer dictionary questions possible within the global budget. Supabase Auth's own limits are unverified. Fix: add per-route @Throttle (e.g. ~30/min on answer submission) and confirm Supabase Auth rate limits in the dashboard.	
F-14 · CONFIRMED · Least Privilege (Read) Teachers see every draft room, not just their own backend/src/catalog/rooms.service.ts:39-43 — draft listings are not scoped to the requesting teacher, feeding discovery for F-01. Fix: scope teacher listings to their own drafts plus all published rooms.	
F-15 · CONFIRMED · PII Minimization Leaderboard falls back to the email local-part as a display name backend/src/leaderboard/leaderboard.service.ts:46 — when no name/username is set, the leaderboard shows email.split('@')[0] , exposing part of a minor's email address to peers. Fix: fall back to a generated pseudonym instead.	
Needs Verification · Backups / Disaster Recovery Backup/PITR status cannot be determined from source render.yaml:19 shows the free plan; no backup configuration exists in-repo; Supabase's free tier has no automatic backups by default. Meanwhile drop_legacy_tables.sql contains seven DROP TABLE ... CASCADE statements (manual/documented, but destructive). Verify: Supabase plan/PITR settings, then perform one test restore. Fix if absent: scheduled pg_dump to separate storage, or upgrade to a plan with backups.	

6 • Verified Secure (No Action Needed)

The following areas were reviewed in depth and found correctly implemented across all three passes. Listed so the team knows what *not* to re-litigate:

AREA	EVIDENCE
Answer grading	<code>submitAnswer</code> returns only <code>isCorrect</code> ; <code>explanation</code> / <code>correctOptionIds</code> are revealed only after a correct answer — the key never reaches the browser beforehand.
Authentication	Supabase JWTs verified with <code>jose</code> against JWKS; <code>issuer</code> and <code>audience=authenticated</code> pinned; legacy HS256 fallback only when explicitly configured; no algorithm-confusion path (<code>jose</code> binds key type to algorithm).
Authorization elsewhere	Notifications, progress, and profile mutations are all keyed to <code>user.id</code> ; class endpoints correctly enforce ownership via <code>assertCanManage</code> — proving the team already knows the right pattern (just hasn't applied it to rooms/paths/modules).
Mass assignment	Global <code>ValidationPipe({ whitelist, forbidNonWhitelisted, transform })</code> ; <code>role</code> / <code>accountStatus</code> / <code>organizationId</code> are deliberately excluded from <code>UpdateProfileDto</code> .
Input validation	All DTOs are class-validator typed with length/range/enum constraints; <code>orderIndex</code> now has <code>@Min(0)</code> (<code>content.dto.ts:58,71,101,110</code>); arrays are bounded with <code>@ArrayMaxSize</code> .
SQL injection	No <code>\$queryRawUnsafe</code> / <code>\$executeRawUnsafe</code> anywhere; the single raw query (health check) uses a parameterized tagged template. Everything else goes through Prisma's query API.
XSS	No <code>dangerouslySetInnerHTML</code> / <code>innerHTML</code> in the codebase; content is rendered via <code>react-markdown@10</code> + <code>remark-gfm</code> , which does not render raw HTML and blocks <code>javascript:</code> URLs by default; external links use <code>rel="noreferrer"</code> .
CSRF	API auth is Bearer-token (no ambient credentials); cookies are <code>SameSite=Lax</code> ; all mutations are POST/PATCH/DELETE; the sole pre-auth GET requires a provider-issued code.
Session/cookie handling	Tokens are never stored in <code>localStorage</code> ; <code>@supabase/ssr</code> manages httpOnly/Secure/SameSite cookies; sign-out calls server-side <code>auth.signOut()</code> , revoking the refresh token.
Secrets hygiene	Only the public Supabase URL and publishable key ever reach the browser (by design); <code>render.yaml</code> marks all secrets <code>sync: false</code> ; a full <code>git log --all</code> scan found no <code>.env</code> files or secret keys ever committed.
Admin surfaces	All <code>/admin/*</code> routes require <code>@Roles(ADMIN)</code> via the global <code>RolesGuard</code> ; no debug/test/metrics routes exist; the only public route is <code>/health</code> .
File uploads / webhooks / SSRF / payments	Not applicable — none of these surfaces exist in the codebase.
Points integrity (elsewhere)	Room-completion bonuses pay out only on first completion outside the race described in F-04; points/XP are never client-writable.

7 · Adversarial Test Scenarios (Static)

Static reasoning only — no live endpoints were called. Runtime confirmation requires operator authorization.

SCENARIO	RESULT
Teacher A accesses/modifies Teacher B's room content	SUCCEEDS — F-01/F-02
Student A reads Student B's progress or notifications	BLOCKED — scoped by <code>profileId</code>
User A accesses a class they don't own/belong to	BLOCKED — <code>assertCanManage</code>
Unauthenticated access to any API route	BLOCKED — global guard, 401 (except <code>/health</code>)
Client injects <code>role: "ADMIN"</code> in a profile update	BLOCKED — ValidationPipe whitelist, field absent from DTO
Student requests teacher role directly	PARTIALLY BLOCKED — lands in PENDING, but see F-05
Client manipulates points/XP/progress values directly	BLOCKED — fully server-computed
Replay of a logged-out access token before natural expiry	POSSIBLE IN PRINCIPLE — standard short-lived JWT trade-off; refresh token is revoked at logout
Brute-force of short-answer questions within rate limit	FEASIBLE — see F-13
Malformed / extra-field payloads	REJECTED — global ValidationPipe
File-upload abuse	N/A — no upload surface exists

8 • Runtime / Infrastructure Checklist

These cannot be proven from source. Someone with dashboard/production access should verify each before launch:

1. **Supabase RLS + Data API** (F-10) — Dashboard → Database → Policies; test each public table with the publishable key.
2. **Supabase Auth limits** — Dashboard → Auth → Rate Limits (login, password reset, email sending) and token lifetime/rotation settings.
3. **Backups/PITR** — confirm plan features, then perform one test restore.
4. **HTTPS + HSTS live** — `curl -sI http://<domain>` should 301→https; check `strict-transport-security` header on both frontend and API domains.
5. **Live security headers** (after F-08 fix) — run a scan against the deployed frontend.
6. **Deployed artifact exposure** — confirm `/.env`, `/.git/config` return 404 on both frontend and API.
7. **Production CORS value** — confirm the Render `CORS_ORIGINS` env contains exactly the production frontend origin(s).
8. **Budget alerts** — Supabase/Render spend notifications configured.
9. **Provider account MFA** — app-based 2FA (not SMS) on Supabase/Vercel/Render/GitHub dashboards.
10. **Log retention** — confirm Render's retention window meets requirements.
11. **CDN caching of authenticated pages** — confirm `cache-control: private, no-store` semantics on signed-in responses.

9 • Remediation Roadmap

P0 — Immediate / release-blocking

Fix	Finding	Size	Breaking risk
Shared ownership guard on every room/path/module authoring endpoint (<code>createdById === user.id ADMIN</code>), incl. validating task↔room nesting	F-01, F-02, F-14	S	Low
Upgrade <code>next@16.3.4</code> + <code>sharp</code> ; then grouped non-breaking <code>npm audit fix</code> on the backend	F-03	S-M	Medium
Partial unique index <code>(profileId, questionId) WHERE is_correct</code> ; derive completion bonus idempotently from the ledger	F-04	S	Low
Verify and lock Supabase RLS/Data API (or disable Data API); drop or lock legacy tables	F-10	S (dashboard)	Low

P1 — Before production

Fix	Finding	Size	Breaking risk
Frontend security headers: HSTS, <code>X-Frame-Options</code> / <code>frame-ancestors</code> , nosniff, Referrer-Policy, Permissions-Policy; CSP Report-Only → enforce after log review	F-08	S	Medium
Teacher least-privilege design: own-content-only scope after approval (depends on the P0 ownership guard); consider a probation flag	F-05	M	Low
Close the <code>safePath</code> backslash bypass; pin redirect origin to <code>NEXT_PUBLIC_SITE_URL</code> instead of <code>x-forwarded-host</code>	F-06, F-07	S	Low
Account deletion: soft-delete with 30-day restore → cascading purge, including the Supabase auth user	F-09	M/L	Low

P2 — Short-term hardening

- Per-route `@Throttle` on answer submission and leaderboard reads; confirm Supabase Auth limits — S, low risk.
- `callback(null, false)` for CORS denials (F-11) — S, low. Generic `P2002` error message (F-12) — S, low. Pseudonym fallback on the leaderboard (F-15) — S, low.
- Add tests: ownership-guard unit/integration coverage, and a concurrency test that reproduces the double-payout race — M, low risk.

P3 — Defense in depth

- Backups/PITR with a tested restore; quarantine the destructive legacy-drop script — S (provider-side), low.
- Audit trail for privileged actions (approve/reject/publish/delete) — M, low.
- Introduce a backend test framework with CI, plus request-ID/structured error references — M, low.

Realistic path to production: every P0 item is small in isolation and none require a redesign. Closing P0 removes both confirmed criticals and the unverified database exposure; closing P0 + P1 addresses everything with a plausible exploitation path today. P2/P3 are hardening, not blockers.

Consolidated from three independent static review passes of `1brahm0f/KiberEduAz` dated 2026-09-11. Read-only method throughout: no files modified, no packages installed, no live infrastructure probed. Findings involving Supabase/Vercel/Render dashboard configuration are marked "needs verification" and cannot be upgraded to "confirmed" without operator-authorized access to those consoles.